Programming 2025/2026

Laboratory

Session 9 (May 12)

String Manipulation & File Handling

First Cycle Degree/Bachelor in Genomics
Dept. Of Pharmacy and Biotechnology (FaBiT)
University of Bologna

String Manipulation

- Several useful methods are defined for manipulating a given string **`s`**
  - **Note**: they do not change the original string, but return a new string (or a list, a Boolean value etc) which must therefore be assigned to a variable
- **`s.split(d)`** returns a list of the words in **`s`**, using **`d`** as the delimiter string

```
>> s = "My name is: Chiara Ferri"
>> s.split()
['My', 'name', 'is:', 'Chiara', 'Ferri']
>> s.split(":")
['My name is:', 'Chiara Ferri']
```

- **`s.isalpha()`** returns **`True`** if all the characters of **`s`** are alphabetic and there is at least one character, **`False`** otherwise

```
>> s.isalpha()
False
```

- **`s.isalnum(), s.isdigit(), s.islower(), s.isupper(), s.isnumeric(), s.isspace(), …`**

String Manipulation

- **`s.upper() / s.lower()`** return a copy of **`s`** with all the cased characters converted to UPPERCASE / lowercase

- **`s.strip(val)`** returns a copy of **`s`** with the leading and trailing characters in the string **`val`** removed. If **`val`** is omitted or **`None`**, it defaults to whitespace

```
>> '   spacious   '.strip()
'spacious'

>> s = '#..... Section 3.2.1 Issue #32 ......'
>> s.strip('.#!')
‘ Section 3.2.1 Issue #32 ‘
```

- For more info, see https://docs.python.org/3/library/stdtypes.html#string-methods and the course slides

File Handling

```
f = open("example.txt", "r")
```

- Opens in reading mode (`r` = read) the file **`example.txt`** (which is located in the same directory as the .py file we are currently working on)

```
f = open("subfolder/example.txt", "r")
```

- Opens a file located in a subfolder of the current folder

```
f = open("/home/joe/example.txt", "r")
```

- Opens a file in a completely different location

File Handling

```
content = f.read()
```

- `content` stores as string the content of the entire file

```
lines = f.readlines()
```

- `lines` contains a lists of all the rows (lines) of the file f (stored as strings)

```
for line in f:
```

- Loops over the lines of a file without storing all the lines
- At each iteration, `line` will contain a row of the file (sequentially read)

```
f.close()
```

- Closes the file at the end of its usage…mandatory…

- For more info, see https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files

File Handling

```
f = open('example.txt', 'w')
f.write('My first text file:\n\n')
f.write('third line, \n')
f.write('fourth line \n fifth line ')
f.write('end of the file')
f.close()
```

`f = open('example.txt', 'w')`

- Opens a file in writing mode '`w`' to write content in the file
- If the file does not exist, creates a new one
- **Otherwise, any writing operation overwrites the content starting from the first line**

`f.write('My first text file:\n\n')`

- Writes content in the file as a string

`f.close()`

- Closes the file (mandatory!)

File Handling

	'r'	'r+'	'w'	'w+'	'a'	'a+'
read	*	*		*		*[1]
write		*	*	*	*	*
create			*	*	*	*
empty the file			*	*		
from the beginning	*	*	*	*		
from the end					*	*

Exercise 11.1

- Write a program that reads a text file and overwrites its content to UPPERCASE. For testing it, create a local text file with a few sentences in it.

Exercise 11.2

- Write a function that copies the content of a given text file into another given text file.

Exercise 11.3

- Write a function that simulates a lottery extraction. Given a list of strings that represent the wheel's names, the function extracts 5 numbers between 1 and 90 (without repetition) for each wheel, and then writes the extraction results in a file.

  E.g., given the list [“Roma”, "Milano", “Bologna”], it writes in a file:

```
Roma: 3 7 88 11 50
Milano: 80 52 73 15 4
Bologna: 13 25 67 18 39
```

Random

```
import random
```

`random.random()`	randomly generates a float number in [0.0, 1.0)
`random.uniform(a, b)`	randomly generates a float number in [a, b]
`random.randint(n, m)`	randomly generates an integer number in [n, m]
`random.randrange(n, m, k)`	**`randomly`** generates an int. number in **`range(n, m, k)`**
`random.choice(s)`	randomly chooses an element of a sequence **`s`**
`random.sample(s, n)`	randomly extracts **`n`** distinct elements from a sequence **`s`**
`random.gauss(mu, sigma)`	randomly generates a float number in a Gaussian distribution. **`mu`** is the mean and **`sigma`** is the standard deviation

- For more info, see https://docs.python.org/3/library/random.html

Exercise 11.4

- Write a program that simulates the hangman game by incrementally solving smaller sub problems.

Exercise 11.4

1. Write a function `find_locations(c,s)` that returns a tuple containing all the indices of a given character `c` in a given string `s`. The returned tuple is empty if no such index exists.

   E.g., `find_locations("p","apple")` returns `(1,2)`

Exercise 11.4

2. Write a program that opens (without losing the original content) a text file **`vocabulary.txt`** and asks the user to input one or more words to the vocabulary. When inputting the words, the user should:

    - separate the words using “;”
    - not use any character with grave accent, acute accent, etc

    The program then should:

    - make sure words contain only alphabetical characters
    - remove any whitespace from the words
    - write each word in the txt file in UPPERCASE, one word per line

Exercise 11.4

3. Continue with the same program to ask the user a user name and to initialise a score to 100. The program then should:

    - reopen the **`vocabulary.txt`** file

    - randomly extract a word and store it in a variable (by removing the end of line character)

    - create a mask (a new string), formed by many dashes (or asterisks, ...) of the same length as the word

Exercise 11.4

4. Continue with the same program to ask the user to guess the word, one letter at a time.
    - This is repeated until the user finds the word or the score reduces to 0.
    - At every guess, the program shows the mask with the correctly guessed characters in position and reads the new letter entered by the user.
        - If the letter is present in the word, the mask is updated highlighting all the occurrences of the letter (via the function `find_locations(c, S)`).
        - If the letter is present but had been guessed previously, this is printed to the user.
        - If the letter is not present in the word, this is printed to the user and a dice is rolled.
            - If the result is not 6, then 10 points are subtracted from the score.
        - At the end of each guess, the current score is printed.

Exercise 11.4

5. Continue with the same program to open or create a text file **`scores.txt`**.

   - In each row of the file there is the score, followed by a space and a username. It does not matter if there are duplicates (a user can play multiple times). E.g.:

     ```
     120 Joe
     92 Anna
     70 Mycol
     65 Mycol
     50 Andrea
     ```

   - The program inserts the current score and the username at the end of the file, only if the file is empty or if the current score is greater than or equal to the minimum score present in the file.

     - (Optional) To open and edit the file without having to close or reopen it, we need to make sure the system starts reading the file from the beginning. Use the **`seek(0)`** method after opening the file in the appropriate mode.